



UNIVERSIDAD DE GRANADA

MusiHub

Nombre del estudiante: ALFONSO FRÍAS FUNES

Tutor/a: Rocío Celeste Romero Zaliz

Grado: GRADO EN INGENIERÍA INFORMÁTICA

Universidad: UNIVERSIDAD DE GRANADA

Departamento: Ciencias de la Computación e Inteligencia Artificial

Curso académico: 2025–2026

TRABAJO FIN DE GRADO

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Motivación	1
1.3. Objetivo del proyecto	2
1.4. Estructura de la memoria	2
2. Estado del arte	4
2.1. Trabajos relacionados	4
2.1.1. Tabla comparativa de soluciones existentes	4
2.1.2. Plataformas comparables	6
2.1.3. Síntesis crítica del análisis comparativo	6
2.1.4. Implicaciones para el diseño del proyecto	6
2.1.5. Conclusiones del estado del arte	7
3. Metodología y planificación	8
3.1. Enfoque de trabajo	8
3.2. Trazabilidad requisitos-planificación	8
4. Análisis y diseño del sistema	9
4.1. Requisitos funcionales	9
4.1.1. Requisitos derivados del estado del arte	9
4.2. Arquitectura técnica del sistema	9
4.2.1. Arquitectura general	9
4.2.2. Cliente móvil (Android)	10
4.2.3. Backend (API)	12
4.2.4. Base de datos	14
4.2.5. Alertas personalizadas y notificaciones	16
4.2.6. Síntesis de decisiones técnicas	18
A. Encuesta a potenciales usuarios	19
A.1. Contexto de la encuesta	19
A.2. Capturas del formulario y resultados	19

A.3. Nota de uso en la memoria	23
--	----

Capítulo 1

Introducción

1.1. Contexto

La comunidad musical actual se apoya cada vez más en entornos digitales para conectar profesionales, difundir proyectos y acceder a oportunidades. En este ecosistema participan perfiles diversos: músicos individuales, bandas, docentes, academias, tiendas de música, salas de conciertos, bares con programación, espacios culturales y otros agentes vinculados a la actividad musical. A día de hoy, gran parte de la interacción entre estos actores se produce mediante redes sociales generalistas, grupos de mensajería y plataformas dispersas (anuncios, eventos, compraventa), donde se publican ofertas, se buscan sustituciones, se proponen colaboraciones o se ofrecen clases y material.

Sin embargo, esta interacción suele estar fragmentada: la información aparece repartida entre múltiples canales, con formatos poco estructurados y con baja capacidad de filtrado. Esto dificulta tareas habituales como: encontrar músicos compatibles para un proyecto, localizar oportunidades laborales puntuales (bolos, sustituciones, clases), organizar propuestas de colaboración o identificar rápidamente material musical en venta dentro de un entorno de confianza.

1.2. Motivación

Aunque existen herramientas digitales para comunicarse y promocionarse, falta una solución especializada que centralice y estructure las oportunidades de la comunidad musical en un único espacio. Las redes sociales generalistas facilitan visibilidad, pero no están diseñadas para hacer *matching* entre necesidades (por ejemplo, una sala que necesita un guitarrista para una fecha concreta) y perfiles (músicos con instrumentos, disponibilidad y ubicación definidas). Del mismo modo, las plataformas genéricas de empleo o compraventa no capturan bien la lógica del sector musical

(sustituciones de última hora, formación de bandas, colaboraciones creativas, clases por especialidad, etc.).

Este TFG se motiva por la necesidad de profesionalizar y agilizar la conexión entre actores musicales mediante una aplicación móvil orientada a: (1) estructurar perfiles (instrumentos, nivel, disponibilidad, ubicación, servicios ofrecidos), (2) facilitar publicaciones (eventos, ofertas, clases, colaboraciones, compraventa) y (3) mejorar la relevancia de la información para cada usuario.

Como elemento diferencial, se propone un sistema de alertas personalizadas, inspirado en el modelo de plataformas de empleo tipo InfoJobs, pero adaptado al contexto musical: el usuario recibe notificaciones cuando aparece una oportunidad coherente con su perfil e intereses (por ejemplo, “batería disponible para banda”, “clases de piano en tu zona”, “sustitución para bolo este fin de semana” o “venta de instrumento del tipo que buscas”).

1.3. Objetivo del proyecto

Objetivo General (OG). Diseñar y desarrollar una aplicación móvil orientada a la comunidad musical que facilite la interacción entre músicos, bandas y entidades del sector, centralizando oportunidades (eventos, clases, ofertas, colaboraciones y compraventa) e incorporando un sistema de alertas personalizadas en función del perfil e intereses del usuario.

1.4. Estructura de la memoria

Antes de entrar en detalle en el desarrollo, se establece la estructura de la memoria de forma coherente con un proyecto de software y consensuada con el tutor/a, para mantener un hilo lógico entre análisis, diseño e implementación. En este tipo de trabajos resulta especialmente útil abordar primero (1) el estado del arte y (2) la metodología, ya que condicionan el alcance, las decisiones técnicas y la planificación del proyecto.

De forma resumida, la memoria se organiza de la siguiente manera:

- **Capítulo 1. Introducción:** contextualiza el dominio, justifica la necesidad del proyecto y define el objetivo general.
- **Capítulo 2. Estado del arte:** análisis de aplicaciones y plataformas comparables dentro del ámbito musical y de modelos tipo empleo/bolsa de oportunidades, recogiendo funcionalidades, ventajas, limitaciones y oportunidades de diferenciación.

- **Capítulo 3. Metodología y planificación:** justificación del ciclo de vida seleccionado (cascada vs. ágil) y descripción del enfoque de trabajo. En este proyecto se parte de un enfoque ágil (Scrum) para iterar en sprints y entregar incrementos funcionales.
- **Capítulo 4. Análisis y diseño del sistema:** requisitos, roles de usuario, arquitectura, modelo de datos y diseño de interfaces.
- **Capítulo 5. Implementación:** desarrollo de la app (funcionalidades principales) y decisiones técnicas relevantes.
- **Capítulo 6. Pruebas y validación:** estrategia de pruebas, resultados y verificación del cumplimiento del objetivo general.
- **Capítulo 7. Conclusiones y trabajo futuro:** conclusiones, limitaciones y posibles mejoras/ampliaciones.

Capítulo 2

Estado del arte

2.1. Trabajos relacionados

2.1.1. Tabla comparativa de soluciones existentes

Como se observa en la Tabla 2.1, la mayoría de soluciones cubren parcialmente las necesidades del dominio (perfiles, anuncios, búsqueda o contacto), pero rara vez integran el flujo completo con alertas realmente personalizadas.

Cuadro 2.1: Comparativa de plataformas relacionadas con la comunidad musical y oportunidades (estado del arte).

Nombre	Tipo	País/alcance	Enfoque	Funcionalidades clave	Matching/alertas	Modelo	Pros	Contras	Link
Vampr	App	Global	Banda / networking	Perfiles de músicos Descubrimiento Chat	Sí: matching tipo swipe + notifs de mensajes	Freemium (suscripción)	Comunidad grande Muy social	Funciones pro de pago Menos foco en jobs	Vampr
BandMix	Web	Global (con .es)	Banda / miembros	Directorio por instrumento/estilo Perfil con media Mensajería	Parcial: avisos/email + mensajería	Freemium (premium para contactar)	Filtros potentes Mucha oferta	Contacto suele requerir pago UX web clásica	BandMix
BuscaMúsicos	Web	España/LatAm	Banda / miembros	Anuncios busco/buscan Perfiles por ciudad/estilo Directorio de salas	No: sin matching algorítmico; búsqueda manual	No confirmado	Orientada a hispanohablantes Simple y directa	Sin app Hub limitado (jobs/market)	BuscaMúsicos
Gigstarter	Web	Europa	Bolos / contratación	Perfiles de artistas Solicitudes de clientes Gestión de reservas	Sí: avisos + recordatorios de eventos	Freemium (plan PRO)	Sin comisiones (según plataforma) Bolos reales	Más cliente->artista que “jobs” Menos comunidad	Gigstarter
Shutapp	Web	España	Jobs + clases + servicios	Clasificados por categorías Geolocalización/radio Perfiles + anuncios	Parcial: feed/últimos anuncios; sin matching automático	Freemium (destacados de pago)	Multi-categoría (similar al TFG) Publicar fácil	Sin app Depende de masa crítica local	Shutapp
Mercasonic (Hispanic)	App/Web	España	Compraventa + músicos + clases	Clasificados (equipo/servicios/empleo) Mensajería Favoritos/seguimiento	Sí: notifs de mensajes + favoritos/seguimientos	Gratis + servicios opcionales / comisiones	Comunidad fuerte Alertas útiles	Más “clasificados” que matching fino Foco gear/foro	Mercasonic
Sounds Market	App/Web	España	Marketplace + clases + salas	Compra/venta Alquiler Clases/salas/talleres	Parcial: avisos de favoritos/búsquedas (según app)	Freemium (comisiones/servicios)	Todo-en-uno musical Nicho claro	Poco foco en bolos/bandas Alcance fuera de ES no confirmado	Sounds Market
EmpleoParaMúsicos	Web	España	Jobs (tablón)	Agrega ofertas/audiciones Categorías por instrumento Publicar ofertas	No: sin perfiles/matching; consulta manual	Gratis/donaciones (no confirmado)	Radar útil de empleo Mucha variedad	No gestiona candidaturas Sin alertas personalizadas	EmpleoParaMúsicos

2.1.2. Plataformas comparables

A partir de la Tabla 2.1, se analizan algunas plataformas representativas para identificar qué patrón funcional validan y qué limitaciones dejan abiertas respecto al objetivo del proyecto.

Vampr (app, alcance global). Valida el interés por perfiles musicales y descubrimiento entre usuarios con interacción social rápida. Su limitación principal es que prioriza el networking frente a la gestión estructurada de oportunidades.

BandMix (web, alcance global). Confirma el valor de combinar perfil rico con filtros por instrumento, estilo y ubicación. Sin embargo, su dinámica es mayoritariamente reactiva (buscar y contactar) y depende en parte de barreras de contacto.

Gigstarter (web, Europa). Aporta una referencia útil para flujos de contratación y gestión de actuaciones. No obstante, su foco está más orientado a contratación puntual que a comunidad multirol con alertas por encaje.

Shutapp (web, España). Es relevante por integrar varias categorías de oportunidad en un único espacio. Su límite es la ausencia de *matching* automático, por lo que el valor depende de la búsqueda manual del usuario.

Mercasonic (web/app, España). Muestra que la comunidad activa y los avisos asociados a seguimiento pueden generar tracción. Aun así, su orientación dominante sigue siendo clasificados y compraventa, con menor recomendación proactiva.

EmpleoParaMúsicos (web, España). Funciona como radar de ofertas y audiciones, útil para centralizar información dispersa. La principal carencia es la falta de perfiles completos, *matching* y flujo de candidatura gestionado.

2.1.3. Síntesis crítica del análisis comparativo

En conjunto, los elementos que se repiten como útiles son perfiles musicales estructurados, filtros por criterios musicales y ubicación, y un mecanismo claro de contacto. También se observa que los avisos básicos (mensajes, favoritos o seguimiento) aportan valor cuando se integran con una comunidad activa.

Las limitaciones comunes son la fragmentación del ecosistema, la ausencia de recomendación proactiva y el ruido generado por publicaciones poco estructuradas o desactualizadas. Este hueco justifica una solución unificada orientada a oportunidades musicales con alertas personalizadas y trazabilidad.

2.1.4. Implicaciones para el diseño del proyecto

Del análisis anterior se derivan decisiones concretas que conectan con los requisitos del Capítulo 4:

- Definir perfiles musicales completos como base del encaje y la búsqueda (RF-01).
- Exigir publicaciones estructuradas por tipo y con campos obligatorios (RF-02).
- Incluir filtros mínimos por ciudad, instrumento y estilo para reducir ruido (RF-03).
- Priorizar alertas proactivas explicables mediante reglas y puntuación (RF-04).
-

2.1.5. Conclusiones del estado del arte

El análisis de plataformas comparables muestra que el valor común se apoya en perfiles musicales estructurados, filtros por criterios musicales y ubicación, y un mecanismo claro de contacto. Sin embargo, se repiten carencias como la fragmentación del ecosistema, la ausencia de matching proactivo y el ruido por anuncios y perfiles poco completos. En respuesta, este TFG plantea una solución unificada que estructura publicaciones por tipo y prioriza un sistema de alertas personalizadas tipo InfoJobs para notificar oportunidades relevantes. Estas conclusiones se traducen en requisitos funcionales y no funcionales que se detallan en el capítulo de análisis, y guían la priorización del MVP y la planificación por sprints.

Capítulo 3

Metodología y planificación

3.1. Enfoque de trabajo

El proyecto se organiza siguiendo un enfoque ágil basado en Scrum, con iteraciones cortas y entregables incrementales orientados al MVP.

3.2. Trazabilidad requisitos–planificación

Trazabilidad requisitos–planificación. Los requisitos derivados del estado del arte se incorporan al *Product Backlog* y se descomponen en tareas implementables priorizadas por valor para el MVP y dependencia técnica. En cada sprint se seleccionan los requisitos de mayor impacto (perfil, publicaciones, filtros y alertas) para obtener incrementos funcionales verificables. La validación del avance se realiza comprobando, al final de cada sprint, el cumplimiento de los requisitos implementados mediante criterios de aceptación y pruebas básicas de los flujos principales (registro de perfil, publicación, búsqueda y recepción de alertas). De este modo, la planificación por sprints queda alineada con los objetivos del proyecto y con las carencias detectadas en las soluciones existentes.

- SPRINTS DE EJEMPLO REVISAR
- Sprint 1: RF-01 + base de RF-02
- Sprint 2: RF-03

Capítulo 4

Análisis y diseño del sistema

4.1. Requisitos funcionales

4.1.1. Requisitos derivados del estado del arte

- RF-01 Perfil musical estructurado. El sistema permitirá crear/editar un perfil con instrumentos, estilos, nivel, ubicación y disponibilidad.
- RF-02 Publicación estructurada por tipo. El sistema permitirá publicar oportunidades clasificadas (clases, bolos/sustituciones, búsqueda de miembros, eventos y compraventa) con campos obligatorios según el tipo.
- RF-03 Búsqueda y filtros mínimos. El sistema permitirá filtrar oportunidades por tipo, ciudad/radio, instrumento/estilo, fecha y texto libre.
- RF-04 Sistema de alertas personalizadas. El sistema permitirá configurar preferencias de alertas (tipos, instrumentos, ubicación y umbrales) y notificará oportunidades que cumplan esos criterios.
-

La mensajería interna se considera deseable por ser común en competidores, pero podrá posponerse a trabajo futuro si excede el alcance del MVP.

4.2. Arquitectura técnica del sistema

4.2.1. Arquitectura general

- **Cliente (Flutter/Android):** interfaz de usuario, consumo de la API, gestión de sesión y recepción de notificaciones.

- **API (FastAPI):** lógica de negocio, autenticación y roles, validación de datos y *endpoints* para perfiles, anuncios y preferencias de alertas.
- **Base de datos (PostgreSQL):** almacenamiento consistente de usuarios, perfiles, anuncios y registros de notificaciones.
- **Proceso de alertas:** evalúa nuevas publicaciones frente a las preferencias de los usuarios mediante *scoring* y registra los envíos.
- **FCM:** canal de entrega de notificaciones *push* hacia la aplicación.

Esta separación permite desacoplar la experiencia móvil del núcleo de negocio y mantener la trazabilidad del sistema de alertas.

4.2.2. Cliente móvil (Android)

Rol del cliente móvil en el sistema

El cliente móvil es la aplicación que utilizan los distintos perfiles de usuario (músicos, bandas y entidades) para interactuar con el sistema. Desde la app se realizan las acciones principales: registro e inicio de sesión, gestión del perfil, publicación y consulta de anuncios, búsqueda y filtrado, configuración de preferencias de alertas y recepción de notificaciones *push*.

Su desarrollo es imprescindible porque el uso central del proyecto se produce en el teléfono móvil y, además, una de las funcionalidades diferenciales del sistema —las alertas— se materializa directamente en forma de notificaciones y en la experiencia de navegación asociada, por ejemplo, accediendo al anuncio relevante al pulsar una alerta.

- Registro e inicio de sesión.
- Gestión del perfil de usuario.
- Publicación y consulta de anuncios.
- Búsqueda, filtrado y navegación por oportunidades.
- Configuración de preferencias de alertas.
- Recepción de notificaciones *push*.

Requisitos técnicos del cliente móvil (Hace falta poner los requisitos aquí?)

Para cumplir con el alcance definido, la aplicación debe cubrir, como mínimo, los siguientes requisitos: (muy largo?)

- Pantallas principales: autenticación, *feed*/listado de anuncios, filtros y búsqueda, detalle de anuncio, crear/editar anuncio, perfil de usuario, preferencias de alertas, favoritos e historial.
- Integración con el backend mediante una API REST.
- Gestión de sesión basada en token y control de acceso en la interfaz (mostrar u ocultar funcionalidades según el estado de autenticación).
- Notificaciones *push* con Firebase Cloud Messaging (FCM), incluyendo la navegación a la pantalla correspondiente al interactuar con la notificación.
- Mantenibilidad: estructura modular del código y posibilidad de incorporar pruebas básicas.

Tecnologías candidatas para el desarrollo de la app

Para implementar el cliente móvil se valoraron tres alternativas habituales en desarrollo móvil: Flutter, Android nativo con Kotlin y React Native. La comparación se realiza atendiendo a criterios de productividad, calidad de integración en Android, mantenimiento y coste de aprendizaje.

Opción A – Flutter (Dart) Flutter permite desarrollar interfaces con alta consistencia visual y un ritmo de implementación elevado, con buen rendimiento y un ecosistema estable para aplicaciones móviles de este alcance.

Opción B – Android nativo (Kotlin + Jetpack) El desarrollo nativo proporciona un control máximo sobre la plataforma Android, aunque suele implicar un mayor tiempo de desarrollo.

Opción C – React Native (JavaScript/TypeScript) React Native ofrece un enfoque multiplataforma apoyado en JavaScript/TypeScript, pero con una integración nativa más dependiente del ecosistema de librerías.

Elección final: Flutter

Tras la comparación, se selecciona Flutter como tecnología principal para el cliente móvil. La decisión se basa en un equilibrio claro entre velocidad de desarrollo y calidad del resultado, especialmente adecuado para una aplicación con varias pantallas, formularios, filtros y un uso intensivo de notificaciones. Además, permite estructurar el proyecto de forma ordenada, lo que facilita tanto el mantenimiento como la trazabilidad en la documentación del TFG.

Decisiones internas dentro de Flutter

Con el fin de reducir riesgos y mantener coherencia técnica, se adopta una organización modular por funcionalidades, una gestión de estado ligera, un cliente HTTP con soporte de autenticación y un almacenamiento seguro de sesión. Los detalles concretos de librerías y paquetes utilizados se describirán en el capítulo de implementación.

4.2.3. Backend (API)

Función del backend

El backend es la parte del sistema que se ejecuta en el servidor. En este proyecto se despliega en local mediante Docker. Su función principal es dar soporte a la aplicación móvil, centralizando la lógica, el acceso a datos y los procesos internos necesarios para el funcionamiento del sistema.

- Exponer una API para que la aplicación móvil desarrollada en Flutter pueda leer y escribir datos (usuarios, perfiles, anuncios, etc.).
- Aplicar la lógica de negocio (validaciones, estados de anuncio, restricciones de edición, etc.).
- Gestionar la seguridad (registro e inicio de sesión, roles y permisos).
- Centralizar procesos automáticos, como el motor de alertas y el envío de notificaciones *push*.
- Conectarse con la base de datos PostgreSQL y garantizar la consistencia de los datos.

Necesidad dentro del sistema

La aplicación no puede limitarse a la interfaz. Para que el sistema sea funcional y evaluable como proyecto de ingeniería, se necesita un backend que permita implemen-

tar, verificar y mantener los flujos principales del producto con un comportamiento coherente y trazable.

- Persistir anuncios y perfiles en una base de datos.
- Definir roles (músico/banda/entidad) y aplicar permisos (solo el propietario puede editar o cerrar un anuncio).
- Ejecutar el *matching* y las alertas de forma fiable y trazable (registro de notificaciones, control de duplicados y de frecuencia).
- Probar la lógica y los *endpoints* con un enfoque de testabilidad (unitarios e integración).
- Generar documentación de *endpoints* con OpenAPI, aportando claridad y valor a la memoria del TFG.

Requisitos del backend para el MVP

El backend del MVP debe cubrir, como mínimo, los siguientes bloques funcionales:

- Autenticación y autorización: registro/login, JWT y roles.
- Gestión de anuncios (CRUD): clases, bolos/sustituciones, miembros, eventos y compraventa, con estados (activo/cerrado/caducado).
- Búsqueda y filtros: ciudad, instrumento, estilo, fecha, texto y precio/remuneración.
- Preferencias de alertas: tipos, instrumentos, estilos, ciudad, umbral y frecuencia (inmediata/diaria/semanal/off).
- Notificaciones: tokens FCM, registro de envíos y mecanismos anti-duplicados/anti-spam.
- Calidad y verificación: tests unitarios (scoring), tests de integración (endpoints) y documentación OpenAPI.

Tecnologías candidatas

Opción A – FastAPI (Python) Framework moderno para APIs en Python, orientado a rapidez y claridad, con documentación OpenAPI automática y buen encaje con la lógica del proyecto.

Opción B – NestJS (Node.js + TypeScript) Alternativa sólida y estructurada, aunque con mayor complejidad inicial para el alcance previsto.

Opción C – Spring Boot (Java) Solución muy robusta y defendible, pero más pesada de configurar para un MVP local y acotado.

Decisión y justificación final

Tecnología elegida: FastAPI (Python).

El proyecto requiere integrar múltiples piezas (autenticación, anuncios, filtros, preferencias, motor de alertas y notificaciones). FastAPI permite avanzar con rapidez sin renunciar a una base técnica sólida. Además:

- La documentación OpenAPI automática aporta valor directo a la memoria por claridad, trazabilidad y facilidad de prueba.
- Python encaja especialmente bien con el componente diferencial del proyecto: el *matching* por reglas y el *scoring*.
- El despliegue en local con Docker permite priorizar productividad, testabilidad y mantenibilidad sin añadir complejidad innecesaria.

4.2.4. Base de datos

Finalidad y papel dentro del sistema

La base de datos es el componente encargado de almacenar la información del sistema de forma persistente, consistente y consultable. En este proyecto, debe guardar los elementos necesarios para sostener la lógica de negocio, la trazabilidad de las alertas y el acceso fiable a la información desde la aplicación móvil y el backend.

- Usuarios y roles (músico, banda o entidad), así como la información asociada a su autenticación.
- Perfiles musicales, con datos relevantes para la comunidad (instrumentos, estilos, nivel, localización, etc.).
- Anuncios de distintos tipos: clases, bolos o sustituciones, búsqueda de miembros, eventos y compraventa.
- Preferencias de alertas, incluida la frecuencia deseada.
- Tokens de dispositivos (FCM) y un registro de notificaciones enviadas, necesario para evitar duplicados y poder evaluar el comportamiento del sistema de alertas.

Su uso resulta imprescindible porque el sistema necesita realizar consultas con filtros, mantener integridad y control de permisos (por ejemplo, quién puede editar o cerrar un anuncio) y conservar un histórico trazable que permita justificar decisiones de diseño desde un enfoque propio de la ingeniería del software.

Requisitos que debe soportar la base de datos

A partir del alcance del proyecto y de los resultados de la encuesta, la base de datos debe permitir:

- Filtrado avanzado de anuncios y perfiles por ciudad, instrumento, estilo, fecha, precio o remuneración y nivel.
- Reducción del “ruido” mediante un modelo que contemple campos obligatorios y estados del anuncio (activo, cerrado o caducado), evitando publicaciones incompletas o inconsistentes.
- Registro de eventos y trazabilidad mediante un *log* de notificaciones (por ejemplo, *notification_log*) que permita evitar el envío repetido de una misma alerta, medir métricas relevantes para el proyecto (cobertura, latencia y relevancia, entendidas como alcance, tiempo de entrega y ajuste al perfil) y facilitar el análisis del funcionamiento del sistema de alertas.
- Soporte para migraciones y evolución controlada del esquema, de forma que los cambios en el modelo de datos puedan mantenerse versionados y trazables.

Tecnologías candidatas

Opción A – PostgreSQL PostgreSQL es una base de datos relacional consolidada, especialmente adecuada cuando se requieren consultas complejas, consistencia transaccional y capacidad de indexación avanzada.

Opción B – MySQL / MariaDB Alternativa relacional madura y ampliamente utilizada, aunque con menos ventajas diferenciales para este caso concreto.

Opción C – Firebase Firestore Base de datos NoSQL gestionada, útil para modelos simples, pero menos adecuada para un sistema con filtros combinados y control relacional explícito.

Elección final: PostgreSQL

Se selecciona PostgreSQL como base de datos principal por su adecuación al tipo de información del proyecto y, especialmente, por su rendimiento en consultas

con filtros múltiples. Además, permite reforzar el rigor del trabajo al facilitar un modelo relacional con restricciones, relaciones y migraciones, y se integra de forma natural en un despliegue local mediante Docker Compose. El soporte de JSONB aporta, además, flexibilidad controlada para campos como instrumentos o estilos si se decide no normalizarlos completamente en una primera versión.

4.2.5. Alertas personalizadas y notificaciones

Objetivo y valor dentro del sistema

Este bloque engloba dos funcionalidades estrechamente relacionadas: la generación de alertas, encargada de identificar qué anuncios son relevantes para cada usuario en función de su perfil y preferencias, y el envío de notificaciones *push*, que comunica al móvil del usuario la existencia de contenido relevante y permite el acceso directo desde la notificación.

Se trata del núcleo diferencial del proyecto: sin este sistema, la aplicación quedaría reducida a un tablón de anuncios tradicional, mientras que las alertas aportan valor al convertir la información en oportunidades personalizadas y accionables.

- Generación de alertas en función de perfil, preferencias y características del anuncio.
- Envío de notificaciones *push* y redirección al contenido relevante dentro de la aplicación.

Requisitos del sistema de alertas

Para que el sistema resulte útil y defendible en un contexto académico, debe cumplir varios requisitos clave:

- *Matching* explicable y auditable: el usuario, y el propio proyecto, deben poder justificar por qué se ha enviado una alerta.
- Control de duplicados: un mismo anuncio no debe notificarse varias veces al mismo usuario.
- Control de frecuencia: el sistema debe permitir notificaciones inmediatas, diarias, semanales o desactivadas.
- Registro e histórico: mantener trazabilidad de los envíos para evaluar el comportamiento del sistema mediante métricas como latencia, cobertura o relevancia aproximada.
- Envío de notificaciones *push* en Android mediante Firebase Cloud Messaging (FCM).

Motor de *matching*: cómo se decide la relevancia

Se propone un modelo basado en reglas ponderadas que asigna una puntuación a cada anuncio según su grado de ajuste al perfil del usuario. Este enfoque permite definir condiciones, por ejemplo, coincidencia de ciudad, instrumento o tipo de anuncio, junto con pesos y un umbral mínimo a partir del cual se genera la alerta.

- Ventajas: *matching* transparente y justificable, implementación rápida y enfoque fácil de validar con escenarios de prueba.
- Limitaciones: no es tan sofisticado como un modelo de aprendizaje automático, pero resulta suficiente para el alcance del proyecto y aporta mayor control y trazabilidad.

Frente a alternativas más complejas basadas en aprendizaje automático, se prioriza este enfoque por su explicabilidad, su viabilidad y su facilidad de validación en el contexto del TFG.

Estrategia de generación: cuándo se crean las alertas

La generación se activa cuando se publica un anuncio. De este modo, los usuarios con frecuencia inmediata reciben una notificación sin esperar a procesos periódicos, y la carga de cálculo se mantiene contenida.

- Ventajas: experiencia más inmediata y menor necesidad de recalcular continuamente.
- Consideración: requiere un control cuidadoso de duplicados y límites para evitar exceso de notificaciones.

Arquitectura de ejecución: proceso auxiliar y planificador

Para evitar mezclar responsabilidades, se plantea ejecutar la lógica de alertas en un componente separado del servidor API.

Se adopta un proceso auxiliar separado con un planificador de tareas, de modo que la API atiende peticiones y el componente auxiliar se encarga de las evaluaciones y envíos. Esta separación mejora la claridad del diseño y evita mezclar responsabilidades.

Envío de notificaciones *push*

FCM es el mecanismo estándar para notificaciones *push* en Android y se integra de forma directa con Flutter y con el backend o *worker*.

- El cliente móvil registra su token FCM y lo envía al backend para asociarlo al usuario.
- El backend realiza el envío con la información necesaria para abrir la pantalla correspondiente desde la notificación.

Otras opciones, como email o SMS, no cumplen el objetivo principal del proyecto y, en el caso del SMS, añaden coste.

Control de ruido y confianza

Los resultados de la encuesta suelen reflejar preocupación por el exceso de notificaciones y el riesgo de contenido no fiable. Por ello, el diseño incluye medidas mínimas para reducir ruido y reforzar confianza:

- `notification_log` para no repetir alertas del mismo anuncio a un mismo usuario.
- Preferencia de frecuencia (inmediata/diaria/semanal/desactivada) y límite máximo diario para evitar saturación.
- Caducidad de anuncios y validación de campos obligatorios para evitar publicaciones incompletas.

4.2.6. Síntesis de decisiones técnicas

Se han seleccionado tecnologías que priorizan productividad y claridad arquitectónica, manteniendo un nivel adecuado de rigor ingenieril. Flutter proporciona una base sólida para el cliente móvil, mientras que FastAPI permite exponer una API REST clara y fácilmente documentable. PostgreSQL aporta consistencia y soporte eficiente para consultas con filtros, necesarias en la búsqueda de anuncios y perfiles. El sistema de alertas se implementa como un proceso auxiliar desacoplado, lo que facilita su evaluación y el control de duplicidades mediante el registro de envíos. La entrega de notificaciones se realiza con FCM. Las decisiones de bajo nivel (librerías concretas y configuración detallada) se describen en el capítulo de implementación.

Apéndice A

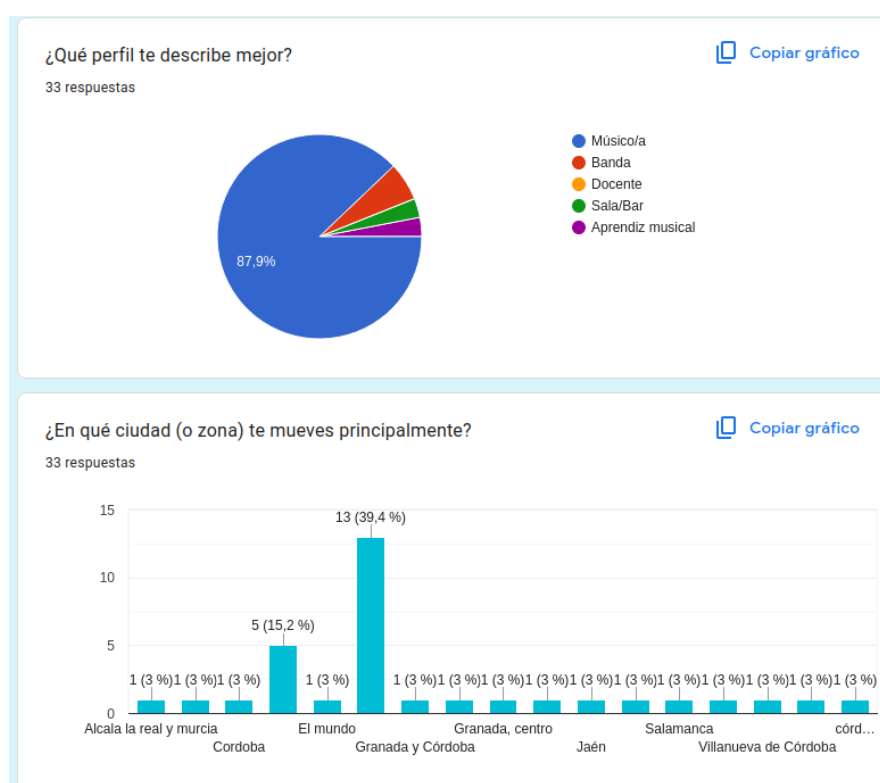
Encuesta a potenciales usuarios

A.1. Contexto de la encuesta

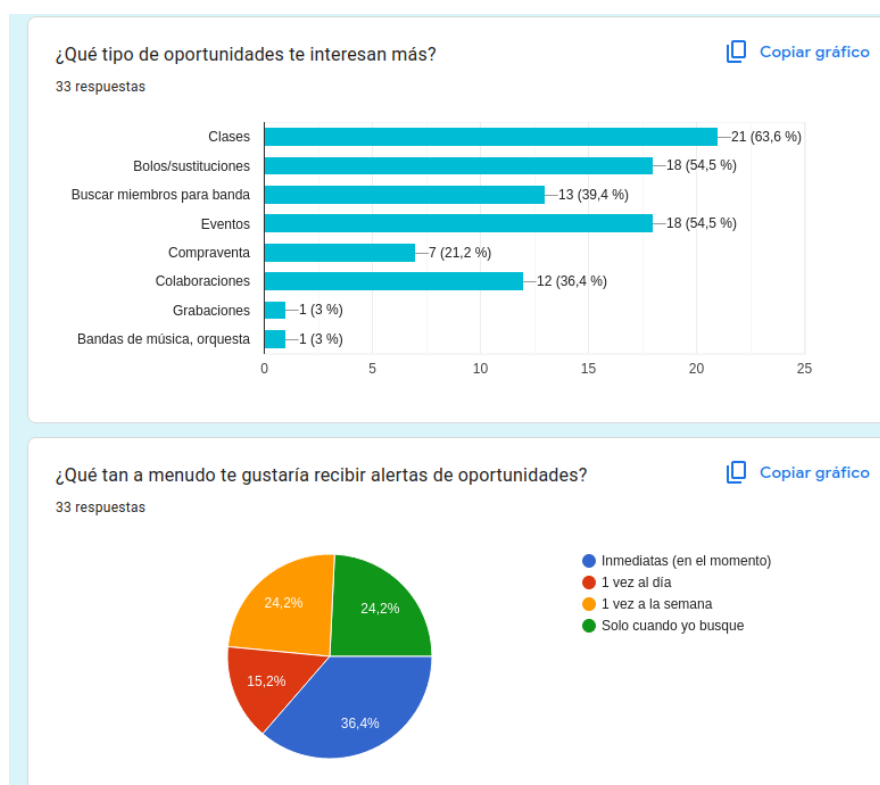
La encuesta se realizó en Google Forms durante febrero de 2026 para recoger preferencias de potenciales usuarios de la aplicación (tipos de oportunidad, filtros, frecuencia de alertas y barreras de uso). Su objetivo fue apoyar decisiones de diseño del sistema con evidencia de uso real.

A.2. Capturas del formulario y resultados

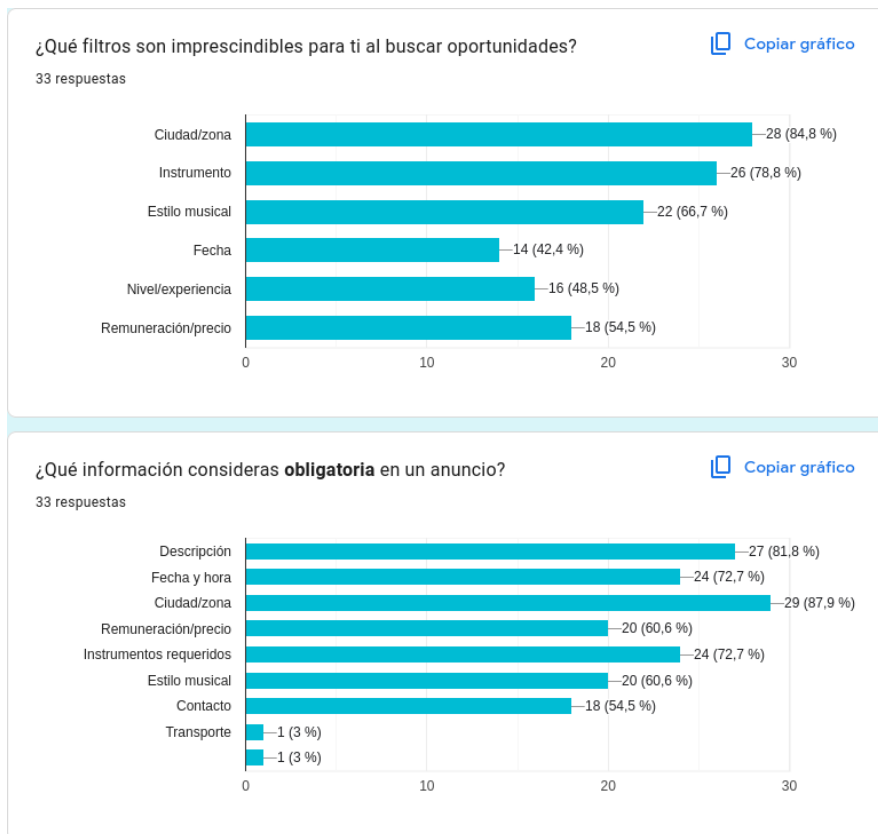
En las siguientes figuras se muestran las capturas del formulario y del resumen de respuestas exportado desde Google Forms.



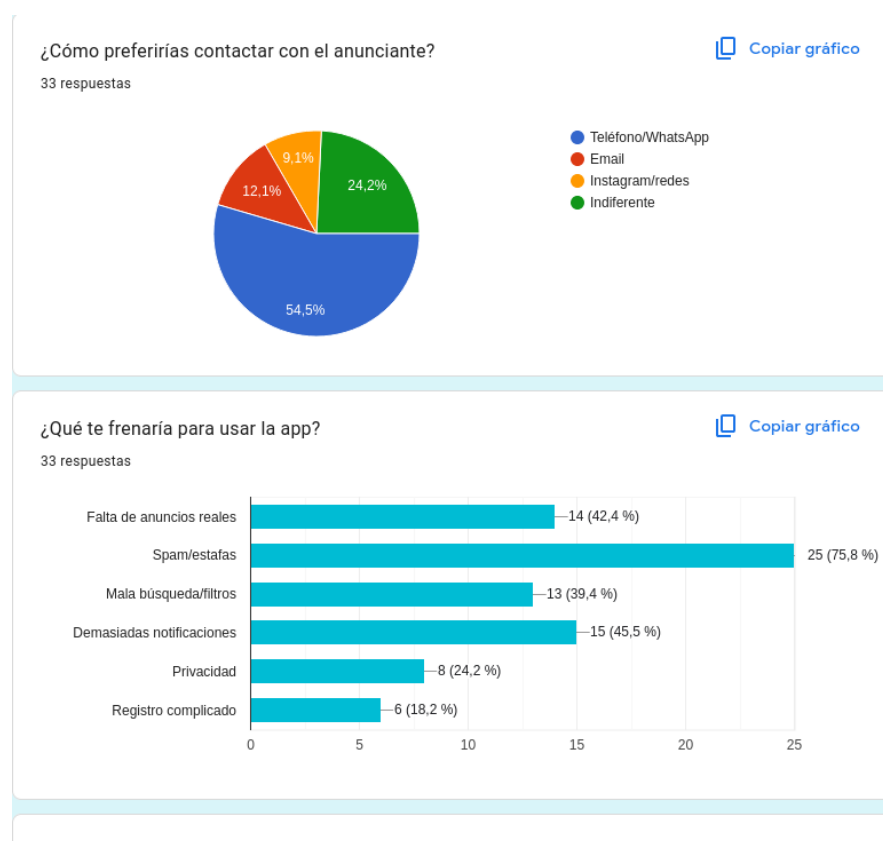
En el perfil de participantes predomina claramente “Músico/a” (87,9 %). En la distribución por zona, la mayor concentración se observa en Granada y Córdoba (39,4 %) y, en segundo lugar, en Córdoba (15,2 %), con el resto de respuestas más dispersas.



Las oportunidades con mayor interés son clases (63,6 %), bolos/sustituciones (54,5 %) y eventos (54,5 %). Sobre frecuencia de alertas, destaca “inmediatas” (36,4 %), seguida de “1 vez a la semana” (24,2 %) y “solo cuando yo busque” (24,2 %).



En filtros imprescindibles, los más votados son ciudad/zona (84,8 %), instrumento (78,8 %) y estilo musical (66,7 %). En la información obligatoria de un anuncio, lideran ciudad/zona (87,9 %), descripción (81,8 %), fecha y hora (72,7 %) e instrumentos requeridos (72,7 %).



El canal de contacto preferido es teléfono/WhatsApp (54,5 %), por encima de email (12,1 %) e Instagram/redes (9,1 %). Como frenos de uso, el principal es spam/estafas (75,8 %), seguido de demasiadas notificaciones (45,5 %) y falta de anuncios reales (42,4 %).

¿Qué función o idea añadirías a esta app que no hayamos mencionado y que te parecería clave?

12 respuestas

- Quizás implementaría también elementos destinados al contacto con técnicos de sonido/visual.
- Ninguna
- Ninguna
- Poder crear tu propio perfil poniendo tu información (instrumento, experiencia, nivel, etc) para poder contactar con otros músicos de la zona deseada
- Búsqueda de salas por estilos
- En los perfiles de los artistas/bandas/profesionales ver donde tienen subidos sus trabajos previos, experiencia, links e incluso formaciones
- Tipo de actividad, profesional o aficionado
- Tocar en la calle

En las respuestas abiertas (12 respuestas) se repiten propuestas como ampliar perfiles (experiencia, nivel, enlaces de trabajo), facilitar búsquedas más específicas por estilo o zona y contemplar necesidades complementarias (p. ej., contacto con técnicos).

A.3. Nota de uso en la memoria

Cuando se cite la encuesta en el cuerpo principal, se referenciará este anexo mediante “Anexo A”.